BRACE: Detailed Integration of Belief Inference and Assistance Arbitration

Supplementary Clarification for Major Weakness 2

Response to THRI-2025-0214 Reviews

1 Overview

This document provides a precise, implementation-grounded specification of how the BRACE framework integrates its Bayesian belief inference module with the reinforcement-learning-based arbitration policy. It is organized around the four specific ambiguities identified by the reviewers:

1. How belief representations are incorporated into action selection (§2).
2. How training data is collected and used across the two training stages (§3).
3. How the framework components (Bayesian inference, expert policy, arbitration policy, simulated human) are integrated into a single pipeline (§4).
4. A step-by-step, variable-level data-flow specification for a single timestep (§5).

Throughout, we use the following notation consistently:

Symbol	Definition
$\mathcal{G} = \{g_1, \ldots, g_N\}$	Set of N candidate goal positions
g^*	True (hidden) goal, unknown to the system
$\mathbf{s}_t \in \mathbb{R}^d$	Environment state (EE position, velocity, relative object/obstacle positions)
$\mathbf{b}_t \in \Delta^{N-1}$	Belief: probability simplex over $\mathcal{G}$ at time t
$\mathbf{c}_t(\mathbf{s}_t)$	Context features extracted from $\mathbf{s}_t$ (minimum obstacle distance)
$\mathbf{h}_t \in \mathbb{R}^2$	Raw human velocity command at time t
$\mathbf{w}_t \in \mathbb{R}^3$	Expert policy action $[v_x, v_y, \text{gripper}]$
$\gamma_t \in [0, 1]$	Assistance blending parameter (output of the arbitration policy)
$\mathbf{a}_t$	Executed action: $\mathbf{a}_t = (1 - \gamma_t)\,\mathbf{h}_t + \gamma_t\,\mathbf{w}_t$
ϕ	Learnable parameters of the Bayesian inference module
θ	Learnable parameters of the PPO arbitration policy

2 How Belief Is Incorporated into Action Selection

The belief vector $\mathbf{b}_t$ influences the executed action through two distinct pathways.

2.1 Pathway 1: Arbitration Policy Input

The arbitration policy π_θ receives a *fused observation* formed by concatenating the environment state $\mathbf{s}_t$ with the full belief vector $\mathbf{b}_t$:

$$\mathbf{o}_t = [\, \mathbf{s}_t \parallel \mathbf{b}_t \,] \;\in\; \mathbb{R}^{d+N} \tag{1}$$

This fused vector is the sole input to the PPO actor-critic network. The actor head outputs a scalar $\tilde{\gamma}_t \in [-1, 1]$ (via tanh activation), which is mapped to the blending parameter:

$$\gamma_t = \tfrac{1}{2}(\tilde{\gamma}_t + 1) \;\in\; [0, 1] \tag{2}$$

Crucially, the policy sees the *entire* probability distribution $\mathbf{b}_t$, not just the MAP estimate $\hat{g} = \arg\max_i b_t(g_i)$. This enables the policy to distinguish, for example, a state where one goal has $p = 0.95$ (high confidence $\Rightarrow$ strong assistance is safe) from a state where three goals each have $p \approx 0.33$ (high ambiguity $\Rightarrow$ assistance risks guiding toward the wrong goal).

The state vector $\mathbf{s}_t$ includes context features $\mathbf{c}_t(\mathbf{s}_t)$—specifically, the minimum distance from the end-effector to the nearest obstacle and the normalized progress toward each candidate goal. These context features, together with $\mathbf{b}_t$, allow the policy to jointly condition assistance on both *intent uncertainty* and *environmental constraint severity*, as required by Theorem 1.

2.2 Pathway 2: Expert Goal Selection at Deployment

The expert policy π_{expert} provides optimal actions toward a *single* goal. During training and deployment, the expert's target is determined differently:

- **Training:** The expert always receives the true goal g^* (via a one-hot encoding appended to its observation). This is possible because the training environment has oracle access to the hidden goal. The expert acts as an ideal oracle: it provides the best possible action *if* the true goal were known with certainty.
- **Deployment:** The true goal is unknown. The expert targets the MAP goal derived from the belief:

  $$\hat{g}_t = g_{\arg\max_i \, b_t(g_i)} \tag{3}$$

  The expert then computes $\mathbf{w}_t = \pi_{\text{expert}}(\mathbf{s}_t, \hat{g}_t)$.

Important clarification: We do *not* compute a belief-weighted mixture of per-goal expert actions. Instead, the belief influences action selection in two complementary ways: (i) γ_t is conditioned on the full belief to modulate *how much* of the expert's action to use, and (ii) the expert's target goal is the MAP estimate from the belief. When belief entropy is high (ambiguous intent), the policy learns to reduce γ_t, which attenuates the expert's potentially incorrect guidance. When the belief concentrates on the true goal, γ_t increases, amplifying correct expert assistance.

2.3 Final Blended Action

Given $\mathbf{h}_t$ (human input), $\mathbf{w}_t$ (expert action), and γ_t (assistance level), the executed action is:

$$\mathbf{a}_t = (1 - \gamma_t)\, \mathbf{h}_t + \gamma_t\, \mathbf{w}_t \tag{4}$$

This linear blending ensures smooth interpolation between full human control ($\gamma_t = 0$) and full expert control ($\gamma_t = 1$). The action $\mathbf{a}_t$ is applied to the environment, producing the next state $\mathbf{s}_{t+1}$.

3 Training Data Collection and Distribution

Training occurs in two sequential stages, each with a distinct data-collection process. Below we describe each stage, the behavior distribution the belief module observes, and how the transition between stages is handled.

3.1 Stage 1: Belief Module Pretraining (Supervised)

Data generation. We generate $M = 5{,}000$ synthetic trajectories of length $T = 100$ timesteps. Each trajectory is produced by the simulated human model operating *without any assistance* ($\gamma = 0$):

1. Sample N goal positions uniformly within the workspace bounds.
2. Randomly select one goal as g^*.
3. Initialize the end-effector at a random starting position.
4. For $t = 1, \ldots, T$:
   - The simulated human computes a deterministic minimum-jerk direction toward g^*.
   - Potential-field obstacle avoidance adjusts the direction near obstacles.
   - AR(1) pink noise (amplitude 3.2%, AR coefficient 0.5) is added to simulate motor variability.
   - The noisy action $\mathbf{h}_t$ is applied directly to the environment (no blending).

Optimization objective. The belief module parameters $\phi = \{\beta, w_\theta, w_d\}$ (constrained positive via softplus) are optimized to minimize the negative log-likelihood of the true goal:

$$\mathcal{L}_{\text{supervised}}(\phi) = -\frac{1}{M}\sum_{m=1}^{M}\frac{1}{T}\sum_{t=1}^{T}\log P_\phi(g^* \mid \mathbf{x}_{1:t}, \mathbf{h}_{1:t}) \tag{5}$$

where the posterior $P_\phi(g^* \mid \cdot)$ is obtained by recursive Bayesian filtering starting from a uniform prior.

Behavior distribution. During pretraining, the belief module observes trajectories generated under *unassisted* human control. The positional trajectory $\mathbf{x}_{1:T}$ reflects the simulated human's noisy goal-directed behavior without expert intervention.

3.2 Stage 2: Joint Arbitration Training (RL + Belief Fine-tuning)

On-policy data collection. The arbitration policy is trained using PPO. At each timestep of each rollout episode, the environment executes the following procedure (Algorithm 1):

1. The simulated human produces $\mathbf{h}_t$ given the *current* EE position and the true goal.
2. The Bayesian module updates $\mathbf{b}_t$ given $\mathbf{h}_t$, the current EE position, and goal positions.
3. The fused observation $\mathbf{o}_t = [\mathbf{s}_t \| \mathbf{b}_t]$ is returned to the PPO policy.
4. The PPO policy outputs γ_t.
5. The expert produces $\mathbf{w}_t$ (toward the true goal during training).

6. The blended action $\mathbf{a}_t = (1 - \gamma_t)\mathbf{h}_t + \gamma_t \mathbf{w}_t$ is executed.
7. The environment transitions to $\mathbf{s}_{t+1}$.
8. The arbitration reward is computed (Eq. 9).

Key distributional considerations.

- **Human model does not adapt to assistance:** The simulated human is a *fixed* model that always computes its action based on the current EE position and the true goal. It does not observe or react to the assistance level γ_t or the expert action $\mathbf{w}_t$. This means the human behavior distribution $P(\mathbf{h}_t \mid \mathbf{x}_t, g^*)$ is identical in both training stages. The belief module's likelihood model $P_\phi(\mathbf{h}_t \mid \mathbf{x}_t, g_i)$ therefore remains well-specified: the same noisy-rational model applies regardless of the assistance context.
- **Positional distribution shifts between stages:** Although the human action distribution is invariant, the *positional* distribution of the EE trajectory $\mathbf{x}_{1:T}$ changes between stages. During pretraining, positions follow unassisted human trajectories. During joint training, the blended action $\mathbf{a}_t$ produces different positional dynamics—typically smoother, more direct paths. The belief module conditions its likelihood on both $\mathbf{h}_t$ and $\mathbf{x}_t$, so it encounters positions it did not see during pretraining. This is precisely why joint fine-tuning (Stage 2) is necessary: it adapts the belief module's parameters to the positional distribution induced by the assistance policy.
- **Belief updates condition on raw human input, not blended action:** The Bayesian module receives the *raw* human velocity command $\mathbf{h}_t$, not the executed blended action $\mathbf{a}_t$. This design choice ensures that the likelihood computation $P(\mathbf{h}_t \mid \mathbf{x}_t, g_i)$ directly models human intent, uncontaminated by the expert's contribution. The blended action only affects the next state $\mathbf{s}_{t+1}$ (and thus the next EE position for the subsequent belief update).

3.3 End-to-End Gradient Flow for Belief Fine-tuning

During Stage 2, the belief module parameters ϕ are jointly optimized alongside the policy parameters θ using a mixed objective that gradually shifts from supervised to reinforcement-driven:

$$\mathcal{L}_{\text{total}}(\theta, \phi) = \alpha\, \mathcal{L}_{\text{RL}}(\theta, \phi) + (1 - \alpha)\, \mathcal{L}_{\text{supervised}}(\phi) \tag{6}$$

where α is annealed from 0.3 to 0.9 over 500,000 timesteps.

The RL gradient with respect to ϕ is estimated via REINFORCE:

$$\nabla_\phi \mathcal{L}_{\text{RL}} \approx \mathbb{E}_\tau \left[\sum_{t=0}^{T} A_t \, \nabla_\phi \log P_\phi(\mathbf{b}_t \mid \mathbf{x}_{1:t}, \mathbf{h}_{1:t}) \right] \tag{7}$$

where $A_t = R_t - V_\theta(\mathbf{s}_t)$ is the advantage estimate (using the PPO critic as baseline).

This gradient flow means that if a particular belief allocation causes the arbitration policy to intervene at the wrong time (increasing task regret), the resulting negative advantage signal pushes the belief parameters to reshape their probability assignments. For example, if the belief module assigns high confidence prematurely in an ambiguous scenario, causing the policy to assist too aggressively toward the wrong goal, the negative reward signal propagates back to adjust β, w_θ, and w_d to produce more cautious (higher-entropy) beliefs in similar states.

Stabilization techniques for this gradient include:

- **Advantage baseline:** The critic $V_\theta(\mathbf{s}_t)$ serves as a state-dependent baseline, centering the REINFORCE gradient to reduce variance.
- **Gradient clipping:** The L2 norm of $\nabla_\phi \mathcal{L}_{\text{RL}}$ is clipped to $C_{\text{clip}} = 1.0$, preserving gradient direction while bounding magnitude.
- **Confidence-scaled updates:** When the maximum belief probability $p_{\max} < c = 0.8$, the update step is damped by a factor $\alpha_{\text{damp}} = p_{\max}/c$, preventing policy oscillation under high belief uncertainty.
- **Temperature annealing:** The softmax temperature τ in the belief update is exponentially decayed from 1.0 to 0.1, gradually sharpening beliefs to provide stronger training signal.

4 Component Integration

BRACE consists of four independently defined components that are integrated through the arbitration training environment. Table 1 summarizes each component's role, inputs, outputs, and trainability.

Table 1: BRACE component specification

Component	**Input**	**Output**	**Training**
Bayesian Inference Module (ϕ)	$\mathbf{h}_t$, $\mathbf{x}_t$, $\{g_i\}$, $\mathbf{b}_{t-1}$	$\mathbf{b}_t \in \Delta^{N-1}$, loss term	Stage 1: supervised NLL; Stage 2: mixed NLL + REINFORCE
Arbitration Policy (θ)	$\mathbf{o}_t = [\mathbf{s}_t \| \mathbf{b}_t]$	$\gamma_t \in [0,1]$, $V(\mathbf{s}_t)$	Stage 2: PPO (actor-critic)
Expert Policy (frozen)	$\mathbf{s}_t$, true goal (train) or MAP goal (deploy)	$\mathbf{w}_t \in [-1,1]^3$	Pre-trained SAC; frozen during Stages 1–2
Simulated Human (fixed)	$\mathbf{x}_t$, g^*, obstacles	$\mathbf{h}_t \in [-1,1]^2$	Not learned; calibrated from human data

4.1 Observation Space Construction

The arbitration policy's observation $\mathbf{o}_t$ is a $(d+N)$-dimensional vector constructed as follows. For the default configuration ($N = 3$ goals, K obstacles):

$$\mathbf{o}_t = [\ \underbrace{\mathbf{x}_t}_{\text{EE pos (2)}} \ \| \ \underbrace{\dot{\mathbf{x}}_t}_{\text{EE vel (2)}} \ \| \ \underbrace{g_{\text{grip}}}_{\text{gripper (1)}} \ \| \ \underbrace{\Delta g_1, \ldots, \Delta g_N}_{\text{rel. goals (2N)}} \ \| \ \underbrace{\Delta o_1, \ldots, \Delta o_K}_{\text{rel. obstacles (2K)}} \ \| \ \underbrace{d^{\text{obs}}_{\min}}_{\text{nearest obs. (1)}} \ \| \ \underbrace{p_1, \ldots, p_N}_{\text{goal progress (N)}} \ \| \ \underbrace{b_t(g_1}_{} \tag{8}$$

where $\Delta g_i = g_i - \mathbf{x}_t$ are relative goal positions, $\Delta o_j = o_j - \mathbf{x}_t$ are relative obstacle positions, and $p_i = 1 - \|g_i - \mathbf{x}_t\|/d_{\text{diag}}$ is normalized progress.

4.2 Arbitration Reward Function

The PPO policy is trained to maximize the following reward, which couples belief quality with task performance:

$$R_t = \underbrace{-w_{\text{coll}} \cdot \mathbf{1}_{\text{collision}}}_{\text{safety}} + \underbrace{w_{\text{prox}} \cdot \gamma_t \cdot p_{\max} \cdot \mathbf{1}_{\text{near}}}_{\text{assist near target}} - \underbrace{w_{\text{far}} \cdot \gamma_t \cdot \mathbf{1}_{\text{far}}}_{\text{penalize premature assist}} + \underbrace{w_{\text{prog}} \cdot p_{\max} \cdot (d_{t-1} - d_t)}_{\text{goal progress}} - \underbrace{w_{\text{auto}} \cdot \gamma_t^2}_{\text{agency preservation}} + \tag{9}$$

where $p_{\max} = \max_i b_t(g_i)$ is the peak belief probability and $p_{\text{true}} = b_t(g^*)$ is the belief assigned to the true goal.

This reward function creates the coupling between belief quality and control:

- The $\log p_{\text{true}}$ term directly rewards accurate goal inference, providing a supervised signal to the belief module during joint training.
- The $p_{\max}$-weighted progress term means that task progress is only rewarded when the belief is confident, encouraging the policy to wait for clear intent signals before assisting.
- The γ_t^2 penalty ensures minimal intervention by default, protecting user agency.
- The proximity and far-distance indicator terms implement context-dependent assistance: help is rewarded near the believed target and penalized far from all targets.

4.3 Curriculum Progression

Training proceeds through five curriculum stages that progressively introduce complexity:

Stage	Name	Goals	Obstacles	Advancement Criterion
1	Basic reaching	1	0	≥100 episodes, success ≥80%
2	Collision avoidance	1	3	≥200 episodes, success ≥75%, collisions ≤15%
3	Challenging obstacles	1	4	≥300 episodes, success ≥70%
4	Goal ambiguity	3	3	≥400 episodes, success ≥65%
5	Full complexity	3	4	Reward plateau (200-episode window)

Stages 1–3 introduce environmental constraints without goal ambiguity, allowing the policy to learn context-aware assistance before facing intent uncertainty. Stage 4 introduces multiple goals, activating the belief module's role in the observation. Stage 5 combines both challenges. This progression ensures stable training: the belief-conditioning pathway is already embedded in the observation from Stage 1 (as a trivially certain 1-goal belief), so the policy gradually learns to use increasingly informative belief signals.

5 Per-Timestep Data Flow

Algorithm 1 specifies the exact sequence of operations at each timestep during arbitration training, making all variable dependencies explicit.

Algorithm 1 BRACE: single-timestep execution during arbitration training

Require: Current state $\mathbf{s}_t$, prior belief $\mathbf{b}_{t-1}$, true goal g^*, trained expert π_{expert}

Ensure: Next state $\mathbf{s}_{t+1}$, reward R_t, updated belief $\mathbf{b}_t$, fused obs. $\mathbf{o}_{t+1}$

— **Human Action** —

$\mathbf{h}_t \leftarrow \textsc{SimulatedHuman}(\mathbf{x}_t,\ g^*,\ \text{obstacles})$ ▷ min-jerk + obstacle avoidance + AR(1) noise

— **Belief Update** —

$\ell_i \leftarrow \exp\big(-\beta\,[w_\theta \cdot \theta_{\text{dev}}(\mathbf{h}_t, \mathbf{x}_t, g_i) + w_d \cdot d_{\text{dev}}(\mathbf{h}_t, \mathbf{x}_t, g_i)]\big)$ for each $g_i \in \mathcal{G}$

$\mathbf{b}_t^{\text{raw}}(g_i) \leftarrow \text{softmax}\Big(\frac{\log b_{t-1}(g_i) + \log \ell_i}{\tau}\Big)$ ▷ Bayes rule with temperature

$\mathbf{b}_t(g_i) \leftarrow \alpha_{\text{ema}} \cdot \mathbf{b}_t^{\text{raw}}(g_i) + (1 - \alpha_{\text{ema}}) \cdot b_{t-1}(g_i)$ ▷ EMA smoothing ($\alpha_{\text{ema}} = 0.85$)

— **Arbitration Policy** —

$\mathbf{o}_t \leftarrow [\mathbf{s}_t \parallel \mathbf{b}_t]$ ▷ fuse state with belief (Eq. 8)

$\tilde{\gamma}_t, V_t \leftarrow \pi_\theta(\mathbf{o}_t)$ ▷ PPO actor: $\tilde{\gamma}_t \in [-1, 1]$; critic: V_t

$\gamma_t \leftarrow \frac{1}{2}(\tilde{\gamma}_t + 1)$ ▷ map to $[0, 1]$

— **Expert Action** —

$\mathbf{o}_t^{\text{expert}} \leftarrow [\mathbf{s}_t \parallel \text{onehot}(g^*)]$ ▷ expert sees true goal (training only)

$\mathbf{w}_t \leftarrow \pi_{\text{expert}}(\mathbf{o}_t^{\text{expert}})$ ▷ frozen SAC or potential-field expert

— **Action Blending and Execution** —

$\mathbf{a}_t \leftarrow (1 - \gamma_t) \cdot \mathbf{h}_t + \gamma_t \cdot \mathbf{w}_t$ ▷ linear blend

$\mathbf{s}_{t+1} \leftarrow \textsc{Environment.Step}(\mathbf{a}_t)$ ▷ apply action, get next state

— **Reward Computation** —

$R_t \leftarrow \textsc{ArbitrationReward}(\gamma_t,\ \mathbf{b}_t,\ g^*,\ d_{t-1},\ d_t,\ d_{\min}^{\text{obs}},\ \text{collision})$ ▷ Eq. 9

— **Store Transition** —

$\mathbf{o}_{t+1} \leftarrow [\mathbf{s}_{t+1} \parallel \mathbf{b}_t]$

Store $(\mathbf{o}_t,\ \tilde{\gamma}_t,\ R_t,\ \mathbf{o}_{t+1},\ V_t)$ in rollout buffer

6 Deployment Data Flow

At deployment on the physical Kinova Gen3 arm (via the ROS node), the data flow changes in two ways:

1. The *human input* $\mathbf{h}_t$ comes from the DualSense controller (or force-sensing joystick) rather than the simulated human.
2. The *expert target* is the MAP goal from the belief ($\hat{g}_t = g_{\arg\max_i b_t(g_i)}$) rather than the true goal.

The deployment pipeline runs at 27 Hz:

1. **Sense:** Read EE pose from `/ee_state`, controller input from `/joy`, object positions from `/object_positions`.
2. **Belief update:** $\mathbf{b}_t \leftarrow \textsc{BayesianInference}_\phi(\mathbf{h}_t, \mathbf{x}_t, \{g_i\}, \mathbf{b}_{t-1})$.
3. **Arbitration:** $\gamma_t \leftarrow \pi_\theta([\mathbf{s}_t \| \mathbf{b}_t])$ using the trained PPO policy (deterministic).
4. **Expert:** $\mathbf{w}_t \leftarrow \pi_{\text{expert}}(\mathbf{s}_t, \hat{g}_t)$ where $\hat{g}_t = \arg\max_i b_t(g_i)$.
5. **Blend and publish:** $\mathbf{a}_t = (1 - \gamma_t)\mathbf{h}_t + \gamma_t \mathbf{w}_t$ to `/cartesian_velocity`.
6. **Transparency:** Publish $\mathbf{b}_t$ and γ_t for the UI overlay (inferred goal highlight, assistance bar).

7 Addressing Specific Reviewer Questions

7.1 R3: How are "raw goal probabilities" used to compute the expert action?

Raw goal probabilities $\mathbf{b}_t$ are **not** used to compute a weighted mixture of goal-conditioned expert actions. The expert produces a single action $\mathbf{w}_t$ toward one goal. During training, this is the true goal g^* (oracle access). At deployment, this is the MAP goal $\hat{g}_t = \arg\max_i b_t(g_i)$.

The belief vector's role is to modulate *how much* of the expert's action to use (via γ_t), not *which* goal the expert targets. When the belief is uncertain (high entropy, $\mathbf{b}_t$ is close to uniform), the policy has learned to output low γ_t, effectively attenuating the expert's contribution regardless of which goal it targets. This is the mechanism by which the full belief distribution—not just the MAP estimate—shapes the executed action.

7.2 R3: How is training data collected for arbitration? Does it reflect closed-loop coupling?

Training data for the arbitration policy is collected on-policy via PPO rollouts. In each rollout, the simulated human, belief module, expert, and policy all execute together in the environment loop (Algorithm 1). The resulting transitions naturally reflect the closed-loop dynamics of the full system: the human acts, the system assists, and the EE moves according to the blended action.

However, the simulated human model is a **fixed, non-adaptive** agent. It computes its action based solely on the current EE position and the true goal, without observing or reacting to the system's assistance level. This means the human's *action distribution* is stationary across episodes, though the *positional trajectory distribution* changes because the blended action $\mathbf{a}_t$ alters how the EE moves.

This is a deliberate modeling choice. In real deployment, users *do* adapt to assistance—but this adaptation is heterogeneous across users and develops over time. By training with a fixed human model, we ensure the policy learns a robust baseline assistance strategy that does not overfit to a particular adaptation pattern. The sim-to-human gap is addressed through: (a) noise calibration from real EEG data (798 trajectories, 23 participants), (b) robustness testing across noise amplitudes (1.5%–5.0%) and spectral slopes (-1.2 to -1.8), and (c) validation via the 12-participant human-in-the-loop study.

7.3 R3: What behavior distribution does the belief model see during pretraining vs. joint training?

- **Pretraining (Stage 1):** The belief module sees trajectories where the EE position $\mathbf{x}_t$ evolves under pure unassisted human control: $\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{h}_t \cdot v_{\max} \cdot \Delta t$. These trajectories are noisier, less direct, and more variable than assisted trajectories.

- **Joint training (Stage 2):** The EE position evolves under the blended action: $\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{a}_t \cdot \Delta t = \mathbf{x}_t + [(1-\gamma_t)\mathbf{h}_t + \gamma_t \mathbf{w}_t] \cdot \Delta t$. These trajectories are smoother and more goal-directed (due to expert contribution), placing the EE in positions the belief module did not encounter during pretraining.

Joint fine-tuning corrects this mismatch: the REINFORCE gradient (Eq. 7) adapts the belief parameters ϕ to produce decision-useful beliefs under the assisted positional distribution. Our ablation (Table 5 in the paper) shows that a frozen pretrained belief achieves only 81.5% success, while warm-started joint training reaches 94.9%, confirming that this adaptation is critical.

7.4 R1: Connection between figures and methodology text

In the revised manuscript, we will replace Figures 1 and 2 with a unified architecture diagram that:

1. Shows every variable ($\mathbf{h}_t$, $\mathbf{b}_t$, $\mathbf{s}_t$, γ_t, $\mathbf{w}_t$, $\mathbf{a}_t$) and its flow direction.
2. Distinguishes the training-time oracle expert path from the deployment-time MAP expert path.
3. Labels the observation construction (Eq. 8) explicitly.
4. Marks which parameters are frozen and which are updated at each training stage.

A TikZ template for the proposed figure is provided in Section 8 below.

8 Proposed Revised Architecture Diagram

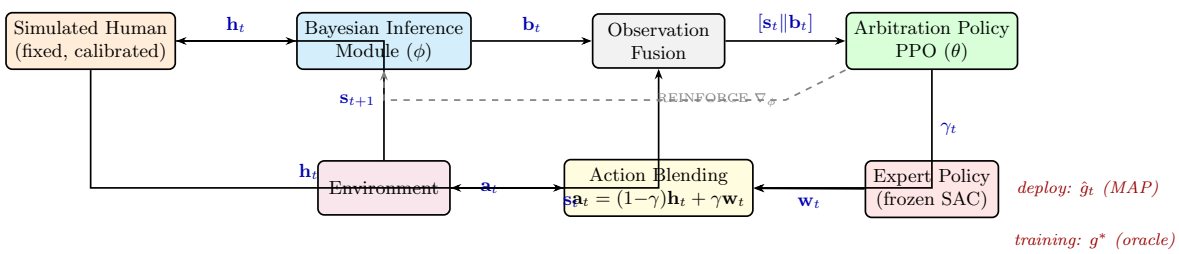

Figure 1: BRACE data flow at each timestep. Solid arrows: forward data flow. Dashed arrow: gradient flow from RL objective back to belief module (Stage 2 only). The expert receives the true goal during training (oracle) and the MAP goal at deployment. The belief vector $\mathbf{b}_t$ is concatenated with the state $\mathbf{s}_t$ to form the arbitration policy's observation.

9 Summary of Changes for Revised Manuscript

Based on the above clarifications, we propose the following specific changes to the manuscript:

1. **Section 3 (Methodology):** Add a formal "Problem Setup and Notation" subsection (before Section 3.1) that defines all variables ($\mathbf{s}_t$, $\mathbf{b}_t$, $\mathbf{c}_t$, $\mathbf{h}_t$, $\mathbf{w}_t$, γ_t, $\mathbf{a}_t$) with their dimensions and domains.
2. **Section 3 (Figures 1–2):** Replace with the unified architecture diagram (Figure 1) that shows all variable flows, distinguishes training and deployment paths, and labels frozen vs. trainable components.
3. **Section 3.1 (Training Procedure):** Expand Algorithm 1 into the detailed form of Algorithm 1, making explicit that (a) the belief conditions on raw human input $\mathbf{h}_t$ (not blended $\mathbf{a}_t$), (b) the fused observation includes the full belief vector, and (c) the expert uses oracle goal access during training.
4. **Section 3.1 (new subsection):** Add "Training Data Collection and Distribution" (content of §3) explaining the two-stage process, the distributional shift between stages, and the fixed-model assumption for the simulated human.

5. **Section 3.3 (Bayesian Inference):** Clarify that the module's three parameters (β, w_θ, w_d) are *learnable* (constrained positive via softplus) and are adapted during joint training.

6. **Section 3.4 (Assistance Arbitration):** State explicitly that the observation is $\mathbf{o}_t = [\mathbf{s}_t \| \mathbf{b}_t]$ (Eq. 8) and that the policy sees the full N-dimensional belief vector, not a scalar summary.

7. **Appendix C:** Expand with the concrete REINFORCE gradient (Eq. 7), the mixed objective (Eq. 6), the α-annealing schedule, and all stabilization techniques.

8. **New Appendix:** Add the deployment data-flow description (§6), making explicit the training-to-deployment differences (MAP goal instead of oracle, real human instead of simulated).